

SWARM TOPOLOGY · 18 MIN READ

Autonomous Multi-Agent Swarms & Hierarchical Orchestration

Engineering High-Throughput Collaborative AI Swarms and Task DAGs

Executive Architecture Summary: Architectural specifications for orchestrating multi-agent networks, hierarchical supervisor-worker patterns, shared vector memory trees, consensus voting mechanisms, and self-healing task execution graphs.

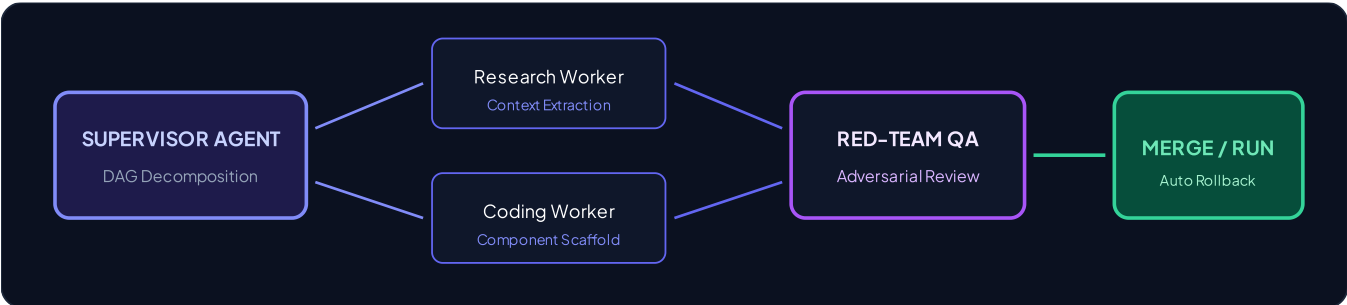
1. Multi-Agent Topology Paradigms

Single-agent systems saturate cognitive limits on complex, multi-stage software workflows. Dividing responsibilities across specialized agent personas—Orchestrator, Research Specialist, Implementation Engineer, Code Reviewer, and QA Validator—drastically reduces error compounding and hallucination cascades.

- Hierarchical Topologies: Master supervisor decomposes objectives into Directed Acyclic Graphs (DAGs) and delegates to workers.
- Peer-to-Peer Consensus: Independent review agents critique proposed implementations in adversarial red-team loops.
- Deterministic Handoffs: Strict JSON schema contracts between agent transitions prevent semantic degradation.
- Context Compression: Intermediate worker findings summarized into executive digests prior to parent integration.

2. Directed Acyclic Graph (DAG) Task Scheduling

Implement deterministic dependency resolution for agent tasks. Independent work nodes execute concurrently in parallel worker threads, while dependent nodes wait for prerequisite state outputs.



3. Shared Memory & Vector State Synchronization

Agent swarms share context across distinct process boundaries using a four-tier memory architecture: L1 Working Scratchpad, L2 Session Transcript, L3 Persistent Vector Store (pgvector / Neon), and L4 Knowledge Graph.

```
// Multi-Agent Task Dispatch Contract Schema
interface AgentTaskNode {
  taskId: string;
  assignedRole: 'architect' | 'frontend' | 'backend' | 'qa_validator';
  dependencies: string[]; // Task IDs that must resolve first
  inputArtifacts: string[];
  maxTokens: number;
  timeoutMs: number;
  retryPolicy: { maxAttempts: 3, backoffMs: 1000 };
  outputSchema: Record<string, any>;
}
```

- State Isolation: Subagents execute within scoped sandboxes without access to unassigned variables.
- Conflict Resolution: When worker suggestions diverge, supervisor computes weighted consensus based on validation test passes.

4. Error Resilience & Self-Healing Swarms

Implement automated circuit breakers for runaway agent executions. If an agent loops on a syntax error or failing test suite, the supervisor captures a snapshot, rolls back to the previous stable git commit, and re-prompts with a failure postmortem.

- Deadlock Detection: Automated supervisor monitors task completion heartbeats every 5 seconds.
- Deterministic Rollbacks: Automated git worktrees isolate risky experimental code changes.